Python Functions

- A function is a block of code that performs a specific task.
- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.

Advantages of Functions in Python

- By including functions, we can **prevent repeating** the same code block repeatedly in a program.
- Python functions, once defined, can be called **many times** and from anywhere in a program.
- If our Python program is large, it can be **separated into numerous** functions which is simple to track.
- The key accomplishment of Python functions is we can return as many outputs as we want with different arguments.

Types of function

1. **Standard library functions** - These are built-in functions in Python that are available to use.
2. **User-defined functions** - We can create our own functions based on our requirements.

Python Function Declaration

```
def function_name(arguments):
    # function body
    return
```

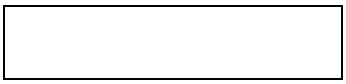
def	The keyword that begins the function definition
function_name	A valid Python identifier that names the function
[parameters]	An optional, comma-separated list of formal parameters for the function
:	The punctuation that denotes the end of the function header
<block>	

Calling a Function in Python

```
def greet():
    print('Hello World!')


# call the function
greet()


print('Outside function')
```



- When the function is called, the control of the program goes to the function definition.
- All codes inside the function are executed.
- The control of the program jumps to the next statement after the function call.

The following are the types of arguments that we can use to call a function:

1. Default arguments
2. Keyword arguments
3. Required arguments
4. Variable-length arguments

Function Arguments

1. **Default Arguments**

A default argument is a kind of parameter that takes as input a default value if no value is supplied for the argument when the function

is called.

```
# defining a function
def function( n1, n2 = 20 ):
   print("number 1 is: ", n1)
   print("number 2 is: ", n2)



# Calling the function and passing only one argument
print( "Passing only one argument" )
function(30)


# Now giving two arguments to the function
print( "Passing two arguments" )
function(50,30)
```

Output:

```
Passing only one argument
number 1 is:  30
number 2 is:  20
Passing two arguments
```

```
number 1 is:  50
number 2 is:  30
```

2. **Keyword Arguments**

```
# Defining a function
def function( n1, n2 ):
   print("number 1 is: ", n1)
   print("number 2 is: ", n2)

# Calling function and passing arguments without using keyword
print( "Without using keyword" )
function( 50, 30)

# Calling function and passing arguments using keyword
print( "With using keyword" )
function( n2 = 50, n1 = 30)
```

3. **Required Arguments**

```
# Defining a function
def function( n1, n2 ):
   print("number 1 is: ", n1)
```

```
  print("number 2 is: ", n2)


# Calling function and passing two arguments out of order, we need num1 to be 20 and num2 to be 30
print( "Passing out of order arguments" )
function( 30, 20 )


# Calling function and passing only one argument
print( "Passing only one argument" )
try:
  function( 30 )
except:
  print( "Function needs two positional arguments" )
```

1. Always put **required arguments first**.
2. Then put **optional arguments with defaults**.

3. **Variable-Length Arguments**

We can use special characters in Python functions to pass as many arguments as we want in a function. There are two types of characters that we can use for this purpose:

- ***args (Variable Positional Arguments)**
    - Collects **multiple positional arguments** into a **tuple**.

- Useful when you don’t know beforehand how many arguments will be passed.

```
def multiply_all(*args):
    result = 1
    for num in args:
        result *= num
    return result

print(multiply_all(2, 3, 4))  # Output: 24
print(multiply_all(5, 10))    # Output: 50
```

- ****kwargs -**These are Keyword Arguments.

- Collects **multiple keyword arguments** into a **dictionary**.
- Useful when you want to handle named arguments dynamically.

```
def print_details(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_details(name="Navin", role="Developer", city="Mumbai")
```

- **Combining *args and **kwargs**

we can use both in the same function, but ***args must come before **kwargs.**

```
def mixed_function(a, b, *args, **kwargs):
    print("a:", a)
    print("b:", b)
    print("args:", args)
    print("kwargs:", kwargs)

mixed_function(1, 2, 3, 4, x=10, y=20)
```

Rule Recap

In Python, function parameters must follow this order:

1. **Positional (non-default) arguments**
2. **Default arguments**
3. *args
4. Keyword-only arguments (after *)
5. **kwargs

Function to a Variable

1. The Basic Assignment

To assign a function to a variable, you simply reference the function name **without** the parentheses ().

```
def greet(name):
    return f"Hello, {name}!"

# Assign the function to a variable
say_hello = greet

# Now you can call say_hello just like greet
print(say_hello("Alice"))  # Output: Hello, Alice!
```

Why does this work?

- In Python, the function name (greet) is actually just a pointer to a function object in memory.
- By saying say_hello = greet, you are telling Python that say_hello should now point to that same memory location.

2. Using Functions in Lists or Dictionaries

```
def add(a, b): return a + b
def multiply(a, b): return a * b
```

```
# Store functions in a dictionary
tools = {
    "plus": add,
    "times": multiply
}

# Access and call the function
result = tools["times"](5, 4)
print(result)  # Output: 20
```